Politeknik Negeri Batam

Machine Learning

Week 9

D4 Robotics Engineering Study Program

Department of Electrical Engineering

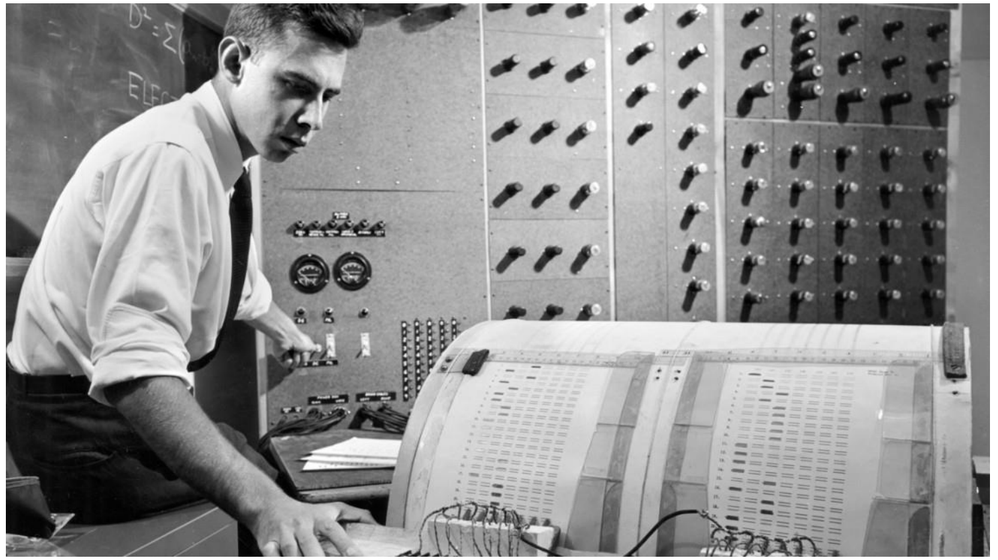
Politeknik Negeri Batam

Neural Network – Single Layer Perceptron

Single Layer Perceptron

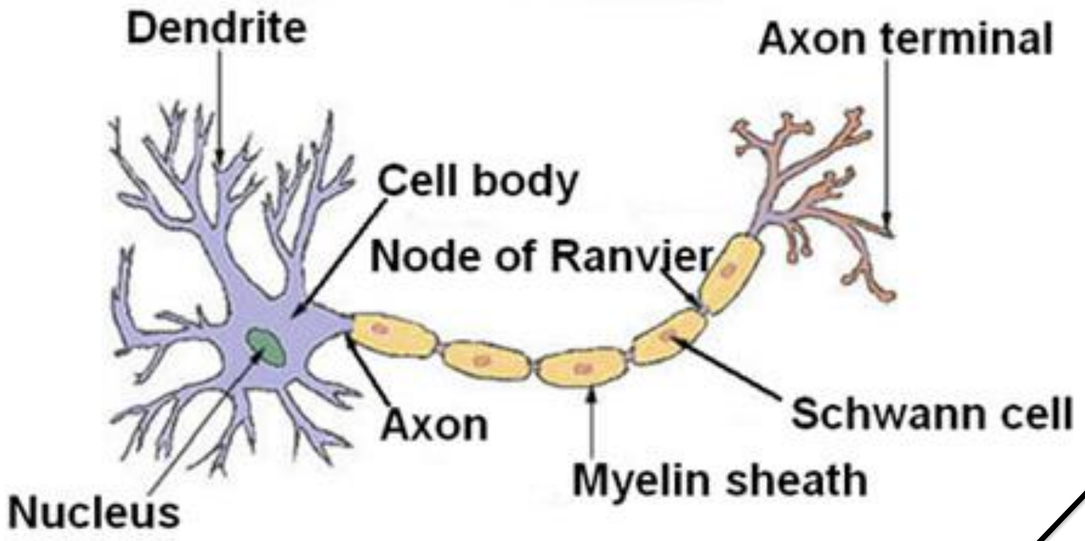
A Brief History

- **1943:** McCulloch & Pitts - Model neuron buatan pertama
- **1957:** Frank Rosenblatt - Perceptron
- **1969:** Minsky & Papert - Keterbatasan perceptron

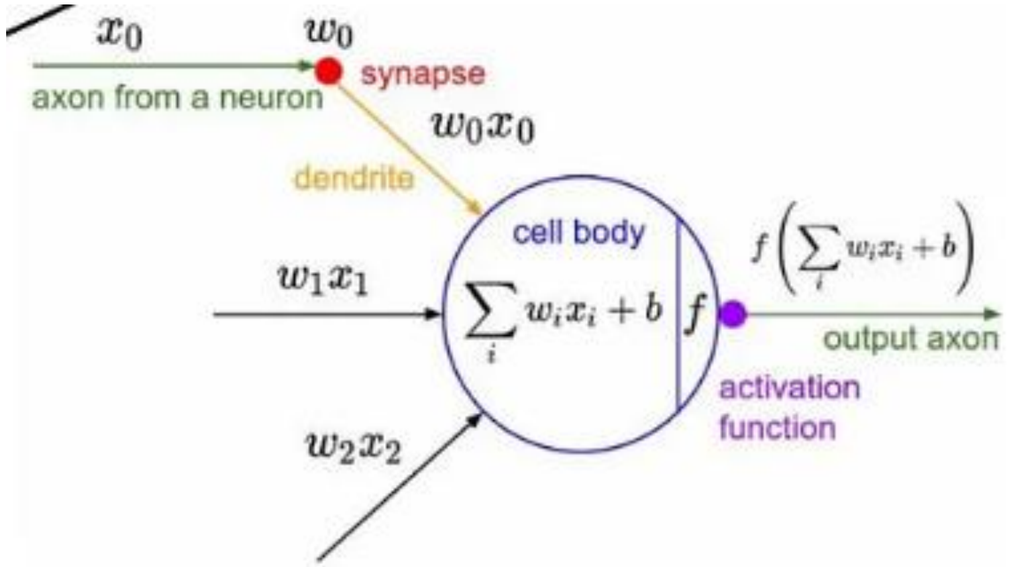


Frank Rosenblatt dan mesin Mark I Perceptron

Neuron in the brain



Artificial Neuron



Neuron in the brain

- **Dendrite:** Menerima sinyal
- **Cell Body:** Memproses sinyal
- **Axon:** Mengirim output
- **Synapse:** Koneksi antar neuron
- **Myelin:** Mempercepat transmisi

Proses: Sinyal elektrokimia → Penjumlahan → Threshold → Fire/Not Fire

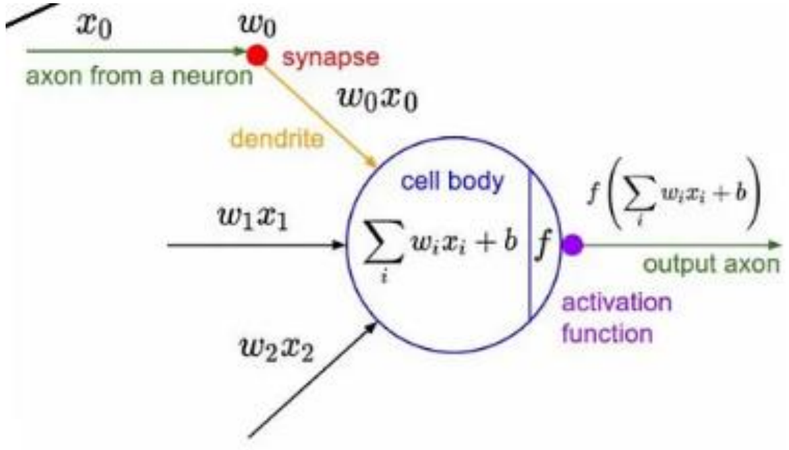
Artificial Neuron

- **Input (xi):** = Dendrite
- **Weights (wi):** = Kekuatan synapse
- **Sum Function:** = Cell body
- **Activation Function:** = Threshold
- **Output:** = Axon

Proses: Input numerik → Weighted sum → Activation → 0/1

The Birth Of The Perceptron

Perceptron: Unit komputasi sederhana yang meniru cara kerja neuron biologis dalam mengambil keputusan biner.



Formula Matematika

$$y = f(\sum(wi \times xi) + b)$$

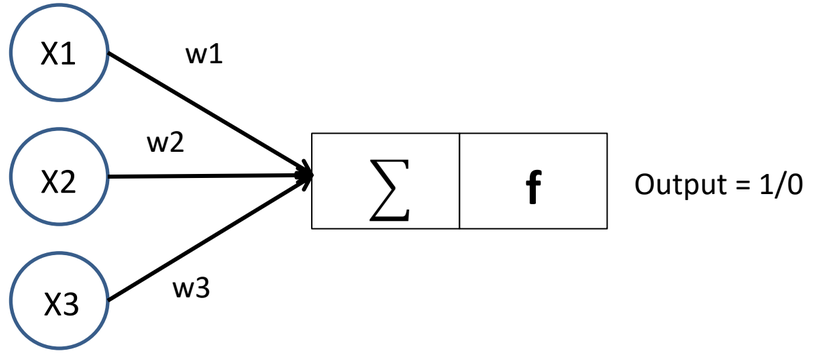
Dimana:

- xi = input ke-i
- wi = bobot ke-i
- b = bias
- f = activation function

Single Layer Perceptron

Single Layer Perceptron adalah model neural network paling sederhana yang terdiri dari:

- Satu layer input
- Satu layer output
- Tidak ada hidden layer



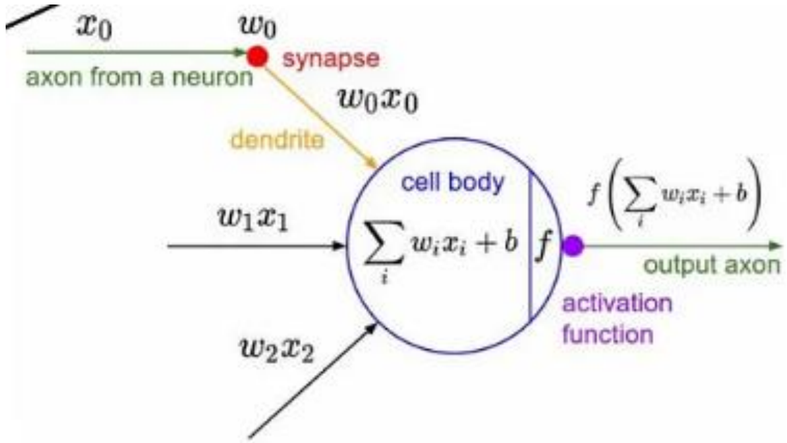
Karakteristik Utama SLP:

- **Linear classifier:** Hanya dapat memisahkan data linear
- **Binary output:** Menghasilkan output 0 atau 1
- **Step function:** Menggunakan fungsi aktivasi tangga
- **Supervised learning**

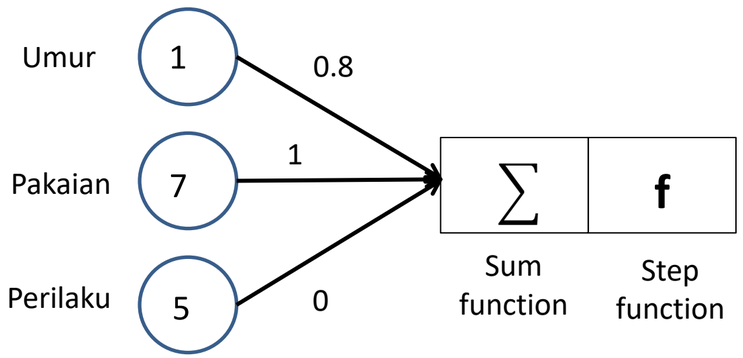
Proses Forward Pass:

Bayangkan seorang guru dikelas menilai siswanya dalam 3 hal yaitu:

1. umur,
2. pakaian dan
3. perilaku.



$$sum = \sum_{i=1}^{n} xi * wi$$



$sum = (1 * 0.8) + (7 * 0.1) + (5 * 0)$

$sum = 0.8 + 0.7 + 0$

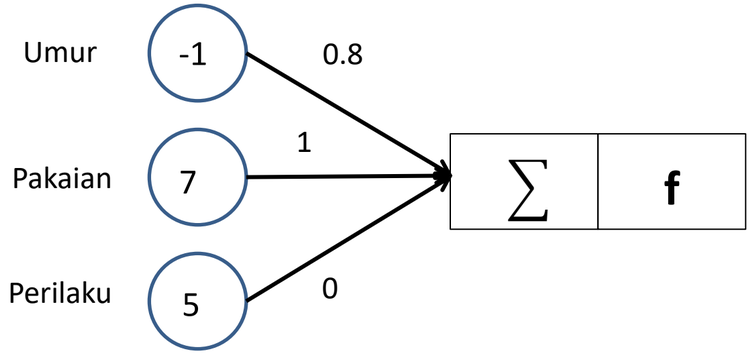
$sum = 1.5$

Output = 1

Step function

Greater or equal to 1 = 1
Otherwise = 0

$$sum = \sum_{i=1}^{n} xi * wi$$



$sum = (-1 * 0.8) + (7 * 0.1) + (5 * 0)$

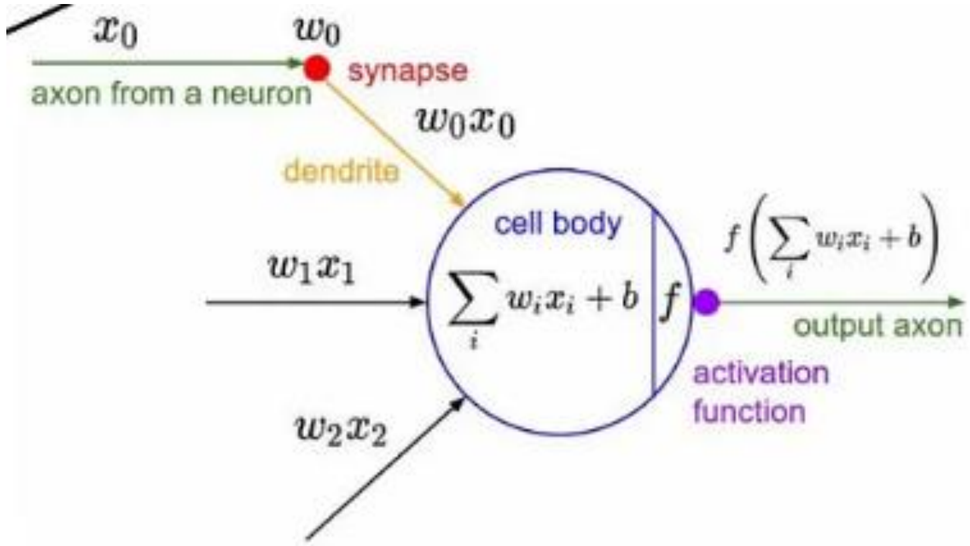
$sum = -0.8 + 0.7 + 0$

$sum = -0.1$

Output = 0

Activation Function - Moment Of Decision

Activation Function mengatur apakah neuron tersebut harus aktif atau tidak.



Step Function

Greater or equal to 1 = 1
Otherwise = 0

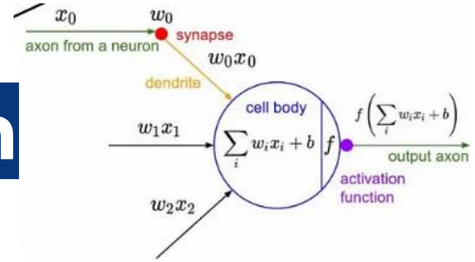
Step Function - Guru tegas:

- Skor ≥ 0: 'MASUK!' (output = 1)
- Skor < 0: 'KELUAR!' (output = 0)

Sigmoid Function - Guru yang lebih halus:

- Skor tinggi: 'Pasti masuk' (output mendekati 1)
- Skor rendah: 'Pasti tidak' (output mendekati 0)
- Skor menengah: 'Hmm... mungkin' (output 0.5)"

Activation Function - Moment Of Decision



- **Sigmoid**

Sigmoid function dikenal juga dengan instilah logistic function, akan menghasilkan nilai pada rentang [0-1], dirumuskan sebagai,

$$f(x) = \sigma(x) = \frac{1}{1 + e^{-x}}$$

- **Rectified Linear Unit (ReLU)**

Rectified Linear Unit (ReLU) function memiliki kelebihan dalam Network yang diinisiasi secara random, hanya 50% dari hidden layer yang akan di aktivasi. ReLU Function di rumuskan sebagai berikut,

f(x) = max (0, x)

atau dalam bentuk rumus, $f(x) = \begin{cases} 0 & \text{for } x \leq 0 \\ x & \text{for } x > 0 \end{cases}$

- **Softmax**

Softmax Funtion atau disebut juga softmax regression adalah bentuk logistic regression yang input valuenya di normalisasi kedalam probability distribution, dirumuskan sebagai,

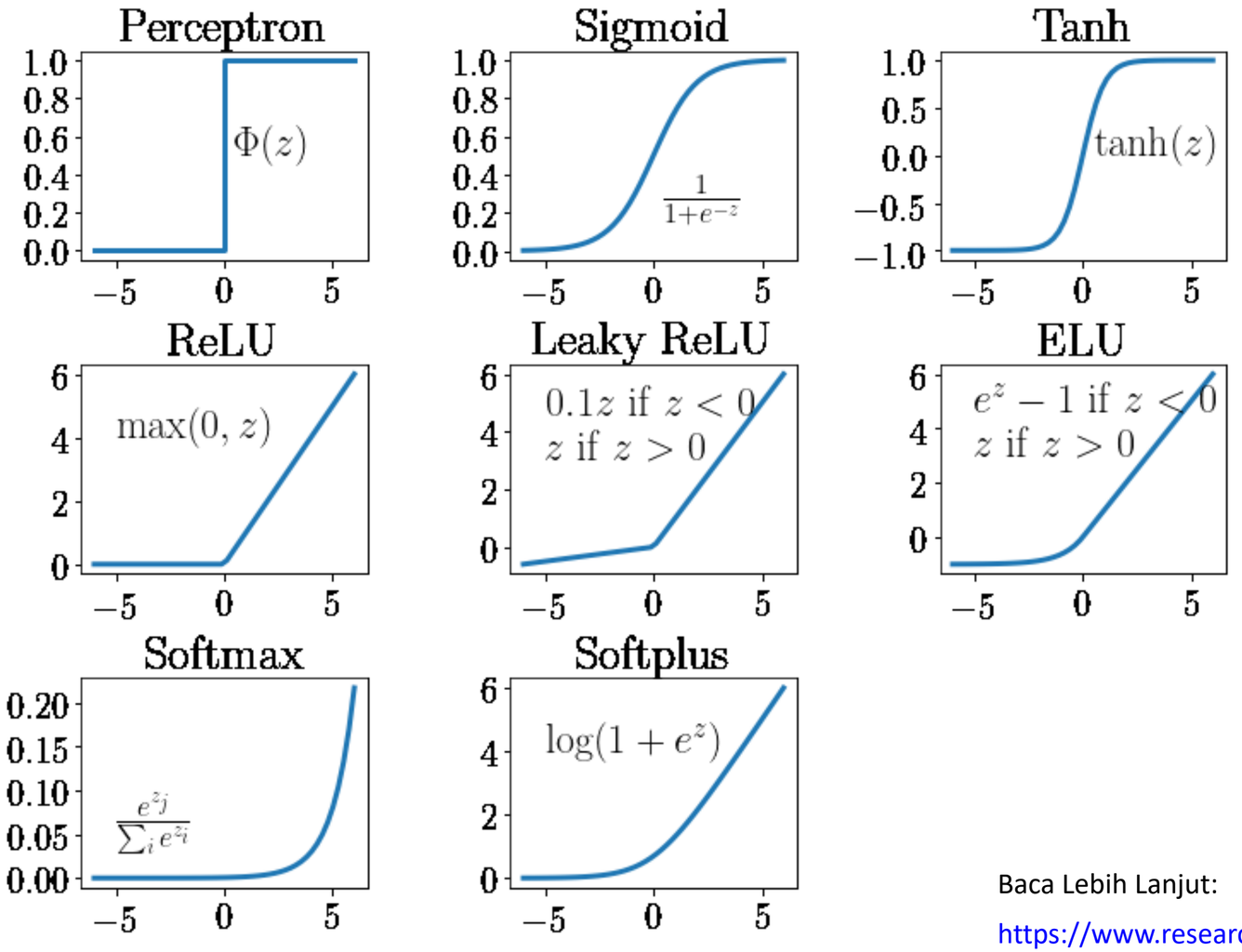
$$f_i(\vec{x}) = \frac{e^{x_i}}{\sum_{j=1}^{J} e^{x_j}}$$

Keuntungan utama dari fungsi ini adalah mampu menangani beberapa kelas.

Baca Lebih Lanjut:

https://medium.com/@cmukesh8688/activation-functions-sigmoid-tanh-relu-leaky-relu-softmax-50d3778dcea5

Activation Function - Moment Of Decision



Baca Lebih Lanjut:

https://www.researchgate.net/publication/341310767_Machine_Learning_for_Materials_Developments_in_Metals_Additive_Manufacturing

Limitations of Single Layer Perceptron

❌ Masalah Utama: XOR Problem

Single layer perceptron **TIDAK DAPAT** menyelesaikan masalah yang tidak linearly separable

✅ Bisa Diselesaikan:

A	B	AND
0	0	0
0	1	0
1	0	0
1	1	1

Linearly Separable - Bisa dipisah dengan garis lurus

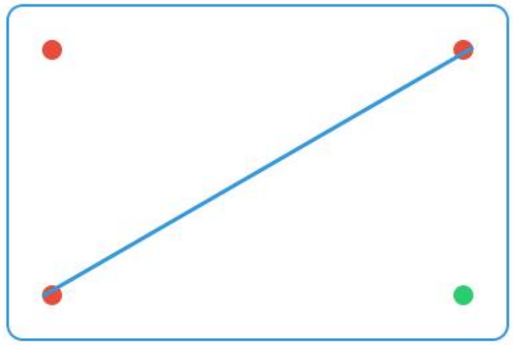
❌ Tidak Bisa Diselesaikan:

A	B	XOR
0	0	0
0	1	1
1	0	1
1	1	0

Non-linearly Separable - Tidak bisa dipisah dengan garis lurus

Decision Boundary Visualization

AND Gate - Linearly Separable



Output = 0 Output = 1

Satu garis bisa memisahkan!

XOR Gate - Non-linearly Separable

Output = 0 Output = 1

Tidak ada garis yang bisa memisahkan!

💡 Solusi untuk XOR:

- **Multi-layer Perceptron (MLP)** - Menambah hidden layer
- **Non-linear activation functions** - Sigmoid, ReLU, dll

Perceptron Learning Algorithm

1. Inisialisasi: Set bobot (w) dan learning rate

```
class Perceptron:
    def __init__(self, num_inputs, learning_rate=0.01):
        self.weights = np.random.rand(num_inputs + 1)  # +1 for bias
        self.learning_rate = learning_rate
        self.errors_per_epoch = []  # For logging errors
```

- **weights**: Termasuk bobot bias (weights[0]).
- **learning_rate**: Seberapa besar perubahan bobot saat training.

2. Fungsi linier

```
def linear(self, inputs):
    return np.dot(inputs, self.weights[1:]) + self.weights[0]
```

Mengalikan input dengan bobot dan menjumlahkan dengan bias.

3. Fungsi Aktifasi

```
def activation(self, z):
    return 1 if z >= 0 else 0
```

Fungsi step: output = 1 jika input ≥ 0, jika tidak output = 0.

4. Prediksi

```
def predict(self, inputs):
    z = self.linear(inputs)
    return self.activation(z)
```

Fungsi step: output = 1 jika input ≥ 0, jika tidak output = 0.

Perceptron Learning Algorithm

5. Update bobot dengan learning rule

```python
def train(self, x_i, y_i):
    prediction = self.predict(x_i)
    error = y_i - prediction
    self.weights[1:] += self.learning_rate * error * x_i
    self.weights[0] += self.learning_rate * error
    return abs(error)  # For tracking
```

- Hitung prediksi → selisih dengan target (error).
- Update bobot dengan rumus:

$$w_i = w_i + \eta \cdot (y - \hat{y}) \cdot x_i$$

- Kembalikan error absolut agar bisa dicatat.

6. Melatih model selama beberapa epoch

```python
def fit(self, X, y, num_epochs=10, verbose=True):
    for epoch in range(num_epochs):
        total_error = 0
        for x_i, y_i in zip(X, y):
            total_error += self.train(x_i, y_i)
        self.errors_per_epoch.append(total_error)
        if verbose:
            print(f"Epoch {epoch+1}/{num_epochs} - Total Error: {total_error}")
```

- Melatih model dengan seluruh dataset selama num_epochs.
- Menyimpan jumlah total error per epoch untuk analisis konvergensi.

7. Mengukur akurasi model

```python
def evaluate(self, X, y):
    predictions = np.array([self.predict(x_i) for x_i in X])
    return np.mean(predictions == y)
```

Practical Application Examples

🔗 Logic Gates

AND Gate:
w1 = 1, w2 = 1, b = -1.5
Threshold = 0

OR Gate:
w1 = 1, w2 = 1, b = -0.5
Threshold = 0

NOT Gate:
w1 = -1, b = 0.5
Threshold = 0

🎯 Real-world Applications

- **Email Spam Detection:**
  Input: word frequencies
  Output: spam/not spam
- **Image Recognition:**
  Input: pixel values
  Output: object class
- **Medical Diagnosis:**
  Input: symptoms
  Output: disease/healthy
- **Credit Approval:**
  Input: financial data
  Output: approve/reject

💡 **Catatan Penting:** Perceptron cocok untuk klasifikasi biner dengan data yang linearly separable!

Conclusion

✅ Kelebihan Perceptron

- Sederhana dan mudah dipahami
- Cepat dalam training
- Guaranteed convergence untuk linearly separable data
- Memori yang dibutuhkan sedikit
- Interpretable weights

❌ Keterbatasan Perceptron

- Hanya untuk linearly separable data
- Tidak bisa solve XOR problem
- Hanya binary classification
- Tidak ada hidden layers
- Linear decision boundary saja

Practice and Assignment

do the practice and do the assignments that are attached in the e-learning

Multi Layer Perceptron-Backpropagation Algorithm

Next Course

Reference

1. https://medium.com/@cmukesh8688/activation-functions-sigmoid-tanh-relu-leaky-relu-softmax-50d3778dcea5
2. https://www.researchgate.net/publication/341310767_Machine_Learning_for_Materials_Developments_in_Metals_Additive_Manufacturing

THANK YOU!